

Spring MVC - raportti

Kuinka Spring MVC toimii?

Spring MVC perustuu Model-View-Controller (MVC) -arkkitehtuurimalliin. Käytännössä kaikki HTTP-pyynnöt kulkevat ensin DispatcherServletin kautta, joka toimii sovelluksen keskuskomponenttina.

DispatcherServlet ohjaa pyynnön oikealle controllerille esimerkiksi @RequestMapping-, @GetMapping- tai @PostMapping-annotaatioiden perusteella. Controller käsittelee pyynnön, hakee tarvittavan datan ja lisää sen Model-olioon. Lopuksi se palauttaa näkymän nimen.

Spring etsii näkymää vastaavan Thymeleaf-template-tiedoston kansiota src/main/resources/templates ja renderöi sen HTML-muotoon käyttäjälle.

Tiedostojen roolit

Spring5mvcApplication.java

Tämä toimii sovelluksen käynnistyspisteenä. Se sisältää @SpringBootApplication-annotaation sekä main()-metodin, joka käynnistää Spring Boot -sovelluksen.

Widget.java

Model-luokka eli tietomalli. Kyseessä on JPA-entiteetti, joka vastaa tietokannan taulua ja määrittelee sovelluksen datarakenteen.

WidgetRepository.java

Repository-rajapinta, joka laajentaa CrudRepository-rajapintaa. Spring Data luo automaattisesti toteutuksen perus CRUD-toiminnoille.

WidgetController.java

Controller-luokka, joka käsittelee HTTP-pyynnöt. Se hakee datan repositorysta, lisää sen modeliin ja palauttaa oikean näkymän.

widgets.html, widget.html, widgetform.html

Näkymät, jotka on toteutettu Thymeleaf-templateina. Ne muuntuvat HTML-sivuiksi ja pystyvät käyttämään modelin dataa.

pom.xml

Mavenin konfiguraatiotiedosto, jossa määritellään projektin riippuvuudet.

Model, View ja Controller

Model

Widget.java toimii tietomallina. Lisäksi Springin Model-olio toimii datan välittäjänä controllerin ja näkymien välillä.

View

Thymeleaf-template-tiedostot muodostavat näkymäkerroksen ja esittävät datan HTML-muodossa.

Controller

WidgetController.java vastaanottaa pyynnöt, käsittelee ne ja ohjaa käyttäjän oikeaan näkymään.

Spring-konteksti Spring Boot -sovelluksessa

Spring Boot luo Spring-kontekstin automaattisesti sovelluksen käynnistyessä ja rekisteröi annotaatioilla merkityt komponentit.

Se konfiguroi myös automaattisesti esimerkiksi Tomcat-palvelimen, H2-tietokannan ja Hibernate EntityManagerin.

@SpringBootApplication-annotaatio

@SpringBootApplication on yhdistelmäannotaatio, joka sisältää kolme osaa:

@Configuration

Määrittelee luokan konfiguraatioksi.

@EnableAutoConfiguration

Käynnistää automaattisen konfiguroinnin.

@ComponentScan

Määrittelee, mistä paketista komponentteja etsitään.

Muiden tiedostojen annotaatiot

@Entity

Merkitsee luokan tietokantaentiteetiksi.

@Id ja @GeneratedValue

Määrittelevät pääavaimen ja sen automaattisen luonnin.

@Controller

Määrittelee controller-luokan.

@Autowired

Mahdollistaa riippuvuuksien automaattisen injektoinnin.

@GetMapping ja @PostMapping

Määrittelevät HTTP-pyyntöjen käsittelyn.

@PathVariable

Mahdollistaa muuttujien lukemisen URL-osoitteesta.